

Week 5

Task 1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency. Develop a program that executes the equivalent of the shell command: `ls-1 | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
root@Tejasri:~# nano week5 1.c
root@Tejasri:~# |
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];
    int data[] = {10, 20, 30, 40, 50};
    int n = 5;
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid > 0) {
        close(pipefd[0]);
        printf("Producer (Parent):\n");
        for (int i = 0; i < n; i++) {
            sprintf(buffer, "%d", data[i]);
            printf("Producing: %d\n", data[i]);
            write(pipefd[1], buffer, strlen(buffer) + 1);
            usleep(100000);
        }
    }
}
```

```

        usleep(100000);
    }
    close(pipefd[1]);
    wait(NULL);
}
else {
    close(pipefd[1]);
    printf("\nConsumer (Child):\n");
    while (read(pipefd[0], buffer, sizeof(buffer)) > 0) {
        printf("Consumed: %s\n", buffer);
    }
    close(pipefd[0]);
}
return 0;
}
|

```

OUTPUT:

```

root@Tejasri:~# nano S5Task1.c
root@Tejasri:~# gcc S5Task1.c -o S5Task1
root@Tejasri:~# ./S5Task1
Producer (Parent):
Producing: 10

Consumer (Child):
Consumed: 10
Producing: 20
Consumed: 20
Producing: 30
Consumed: 30
Producing: 40
Consumed: 40
Producing: 50
Consumed: 50
root@Tejasri:~# |

```

Task 2

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
root@Tejasri:~# nano S5Task2.c
root@Tejasri:~# |
```

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int pipefd[2];
    pid_t pid1, pid2;
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }
    // Create first child
    pid1 = fork();
    if (pid1 == -1)
    {
        perror("fork");
        exit(1);
    }
    if (pid1 == 0)
    {
        // First child: ls -l
        close(pipefd[0]);
        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp ls");
    }
}
```

```

// Create second child
pid2 = fork();
if (pid2 == -1)
{
    perror("fork");
    exit(1);
}
if (pid2 == 0)
{
    // Second child: grep ".c"
    close(pipefd[1]);
    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);
    execlp("grep", "grep", ".c", (char *)NULL);
    perror("execlp grep");
    exit(1);
}
// Parent process
close(pipefd[0]);
close(pipefd[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
printf("Command executed successfully.\n");
return 0;
}

```

OUTPUT:

```
root@Tejasri:~# nano S5Task2.c
root@Tejasri:~# gcc S5Task2.c -o S5Task2
root@Tejasri:~# ./S5Task2
Dashboard.c
MyShell.c
Ptask1.c
Ptask2.c
S5Task1.c
S5Task2.c
Session3.c
Task1.c
Task1.c.save
Task2.c
Task2.c.save
Task3.c
Task4.c
Week3.c.save
copy.c
docs
filename.c
include
sTask.c
sTask1.c
sTask2.c
sTask3.c
sTask4.c
scripts
session3.c
shell.c
shopping.c
shopping.c.save
skill.c
source.txt
```

```
skill.c
source.txt
src
task3.c
week1.c.save
week2.c
Command executed successfully.
root@Tejasri:~# |
```